IMPORTING LIBRARIES

In [1]:
```
import numpy as npimport pandas as pd
```

In [2]:
```
import matplotlib.pyplot as pltimport seaborn as sns%matplotlib inlineimport
w
```

READING THE DATA

In [3]:
```
superster_sales = pd.read_excel('US Superstore
data.xls')superster_sales.sampl
```

Out[3]:

	RowID	Order ID	OrderDate	Ship Date	ModeShip	CustomerID	CustomeNam
	3167	3168 CA-2016-146682	2016-10-29	2016-10-31	Class First	KW-16435	Willma Katrin
	524	525 US-2016-139486	2016-05-21	2016-05-23	First Class	LH-17155	Log Haushal
	7159	7160 CA-2015-145835	2015-05-13	2015-05-18	Second Class	BF-11170	Ben Ferr
Tob	7500	7501 CA-2017-121048	2017-07-14	2017-07-18	Standard Class	TC-21295	Carlis
And	2152	2153 CA-2015-123505	2015-11-22	2015-11-26	Standard Class	AR-10540	Reit

5 rows × 21 columns

EXPLORATORY DATA

In [4]:
```
df = superster_sales.copy()
```

In [5]:
```
df.shape
```

Out[5]:
```
(9994, 21)
```

In [6]:
```
df.columns
```

Out[6]:
```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',       'C
ustomer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',       'P
ostal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',       'Produ
ct Name', 'Sales', 'Quantity', 'Discount', 'Profit'],      dtype='object')
```

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  ----0
 Row ID          9994 non-null   int64
  1  Order ID       9994 non-null   object
  2  Order Date     9994 non-null   datetime64[ns]
  3  Ship Date      9994 non-null   datetime64[ns]
  4  Ship Mode      9994 non-null   object
  5  Customer ID    9994 non-null   object
  6  Customer Name  9994 non-null   object
  7  Segment        9994 non-null   object
  8  Country        9994 non-null   object
  9  City           9994 non-null   object
  10 State          9994 non-null   object
  11 Postal Code    9994 non-null   int64
  12 Region         9994 non-null   object
  13 Product ID     9994 non-null   object
  14 Category       9994 non-null   object
  15 Sub-Category   9994 non-null   object
  16 Product Name   9994 non-null   object
  17 Sales          9994 non-null   float64
  18 Quantity       9994 non-null   int64
  19 Discount       9994 non-null   float64
  20 Profit         9994 non-null   float64 dtypes: datetime64[ns](2),
float64(3), int64(3), object(13) memory usage: 1.6+ MB
```

In [8]:
```
df[['Sales','Discount','Profit','Quantity']].describe()
```

Out[8]:

	Sales	Discount	Profit	Quantity
count	9994.000000	9994.000000	9994.000000	9994.000000
mean	229.858001	0.156203	28.656896	3.789574
std	623.245101	0.206452	234.260108	2.225110
min	0.444000	0.000000	-6599.978000	1.000000
25%	17.280000	0.000000	1.728750	2.000000
50%	54.490000	0.200000	8.666500	3.000000
75%	209.940000	0.200000	29.364000	5.000000
max	22638.480000	0.800000	8399.976000	14.000000

drop unnecessary columns

In [9]:
```
df=df.drop('Row ID',axis=1)
```

In [10]:
```
df['Country'].value_counts()
```

Out[10]: CountryUnited States 9994Name: count, dtype: int64

In [11]:
```
df=df.drop('Country',axis=1)
```

In [13]:

In [14]:
```
df.drop(['Order ID', 'Ship Date', 'Ship Mode', 'Cust
```

Out[14]:

```
df.shape
```

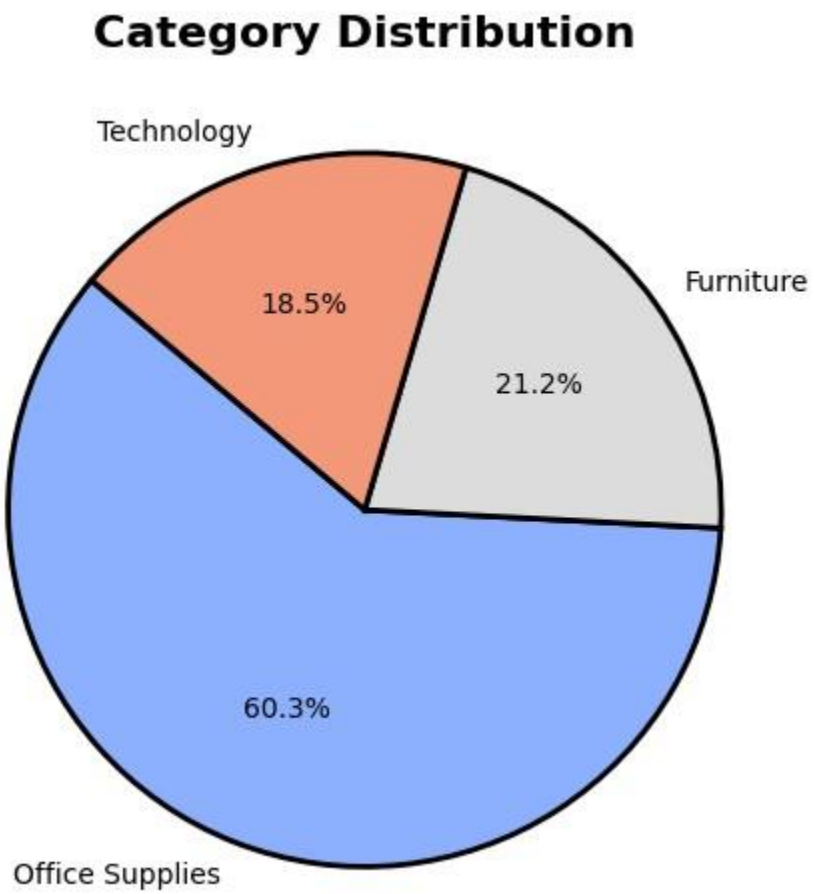
In [15]: (9994, 11) review the product categories available

Out[15]:

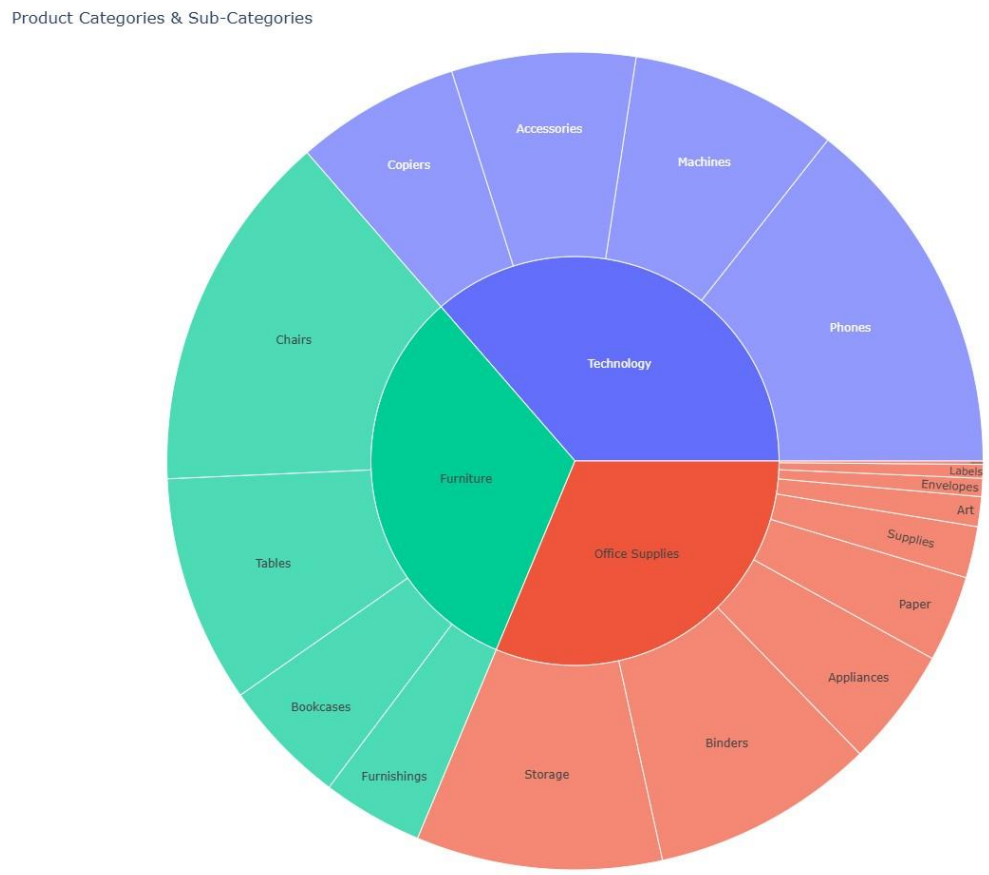
```
category_counts = df['Category'].value_counts()category_counts
```

CategoryOffice Supplies 6026Furniture 2121Technology 1847
Name: count, dtype: int64

In [16]:
```
fig, axs = plt.subplots(figsize=(5, 5))axs.pie(category_counts, lab
```



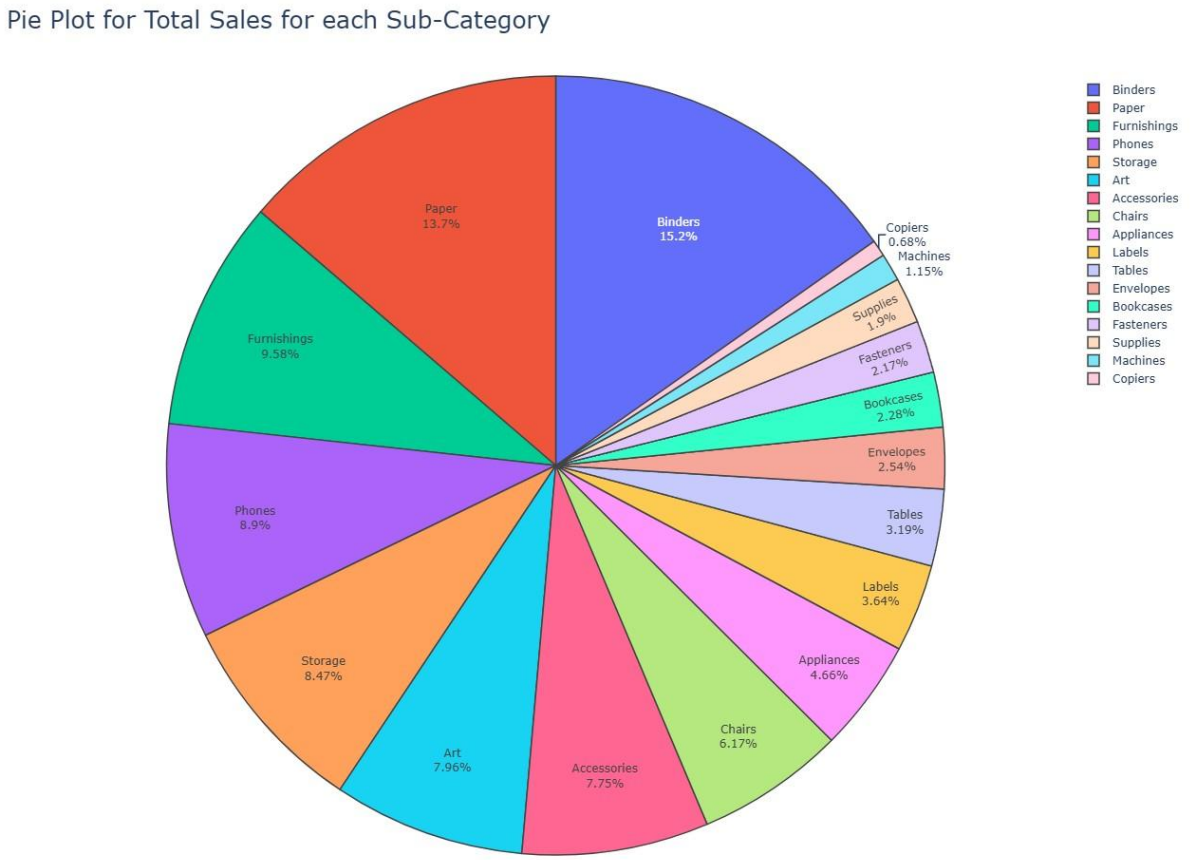
In [41]:
```
import plotly.express as pxfig = px.sunburst(df,path=['Category','Sub-Category
```



Above graphs indicates->Share of each category & sub-category in Sales->Which sub-category belongs to which category->Share of sub-category for each category(click on any category to get detailed view of each sub-category)

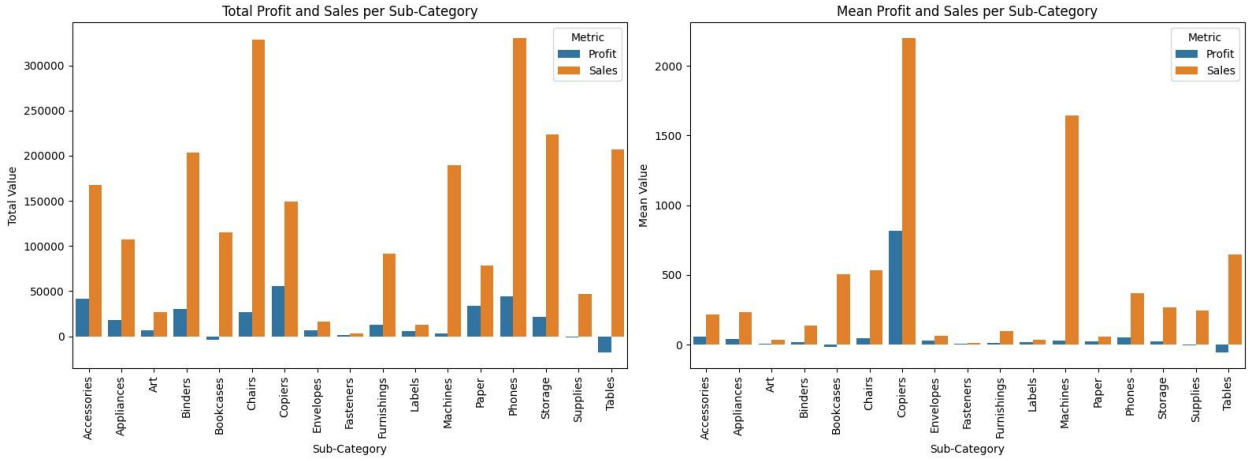
You may get a wrong view of how many products are soldtotal sales in pie plot

In [19]:
```
import plotly.graph_objects as gofrom plotly.offline import iplotdata=go.Pie(
```

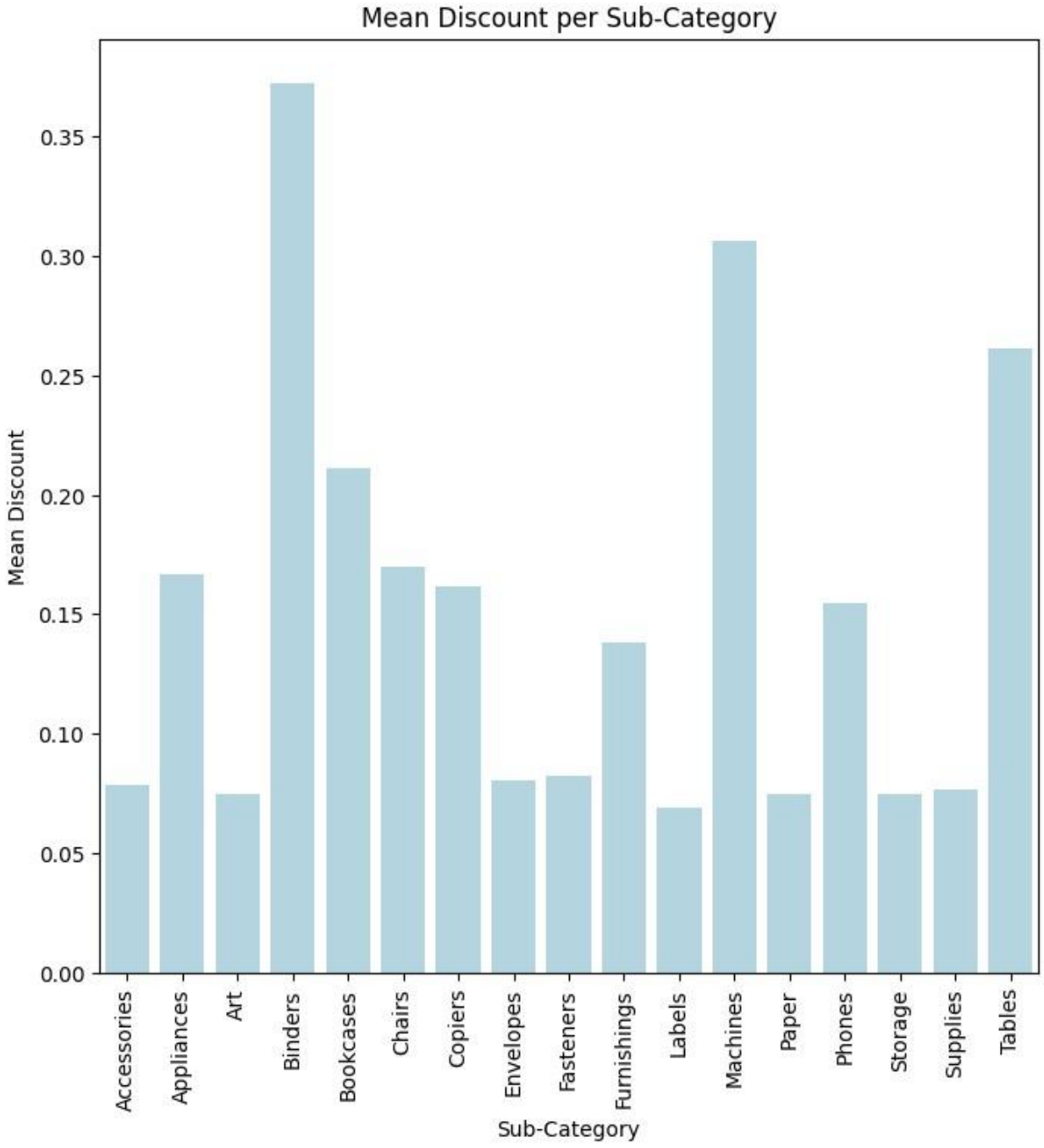


Review of sales and profits

In [20]:
```
grouped_sum = df.groupby('Sub-Category')[['Profit', 'Sales']].sum().reset_inde
```



In [21]:
```
grouped_discount_mean = df.groupby('Sub-Category')['Discount'].mean().reset_in
```



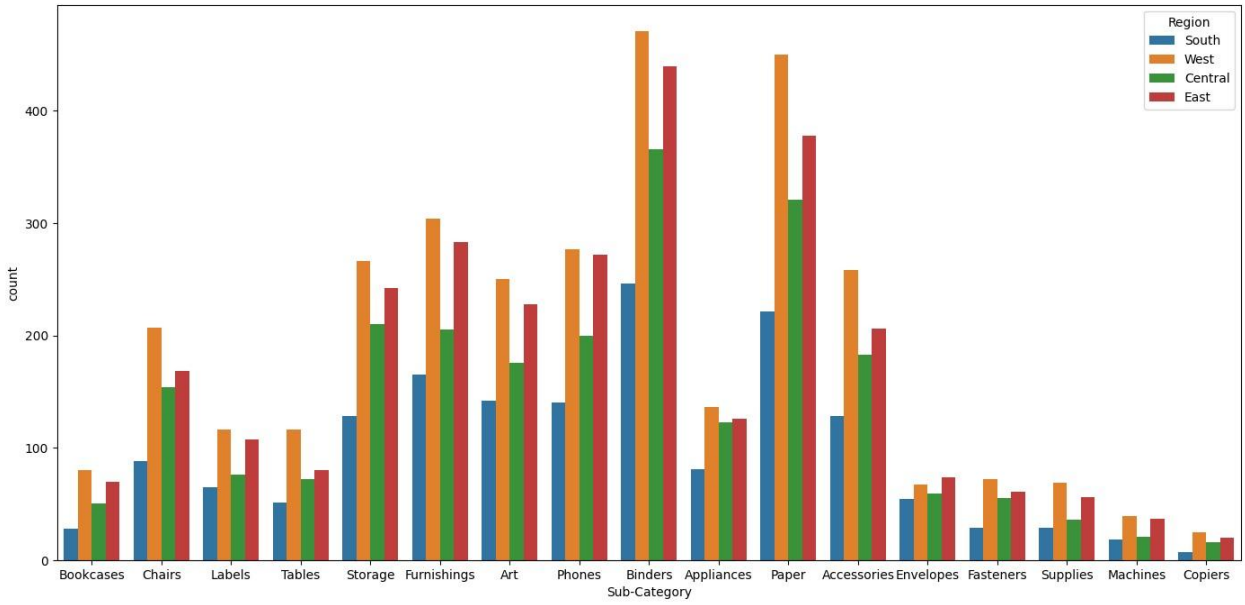
The store's discount system has a problem

->Best-selling products can be sold with no discount or with a partial discount>Products that are causing damage to the store should be sold without discount>Products with limited sales and higher profitability should be sold at a higher discount to attract customers

review of the sales of different parts of the US

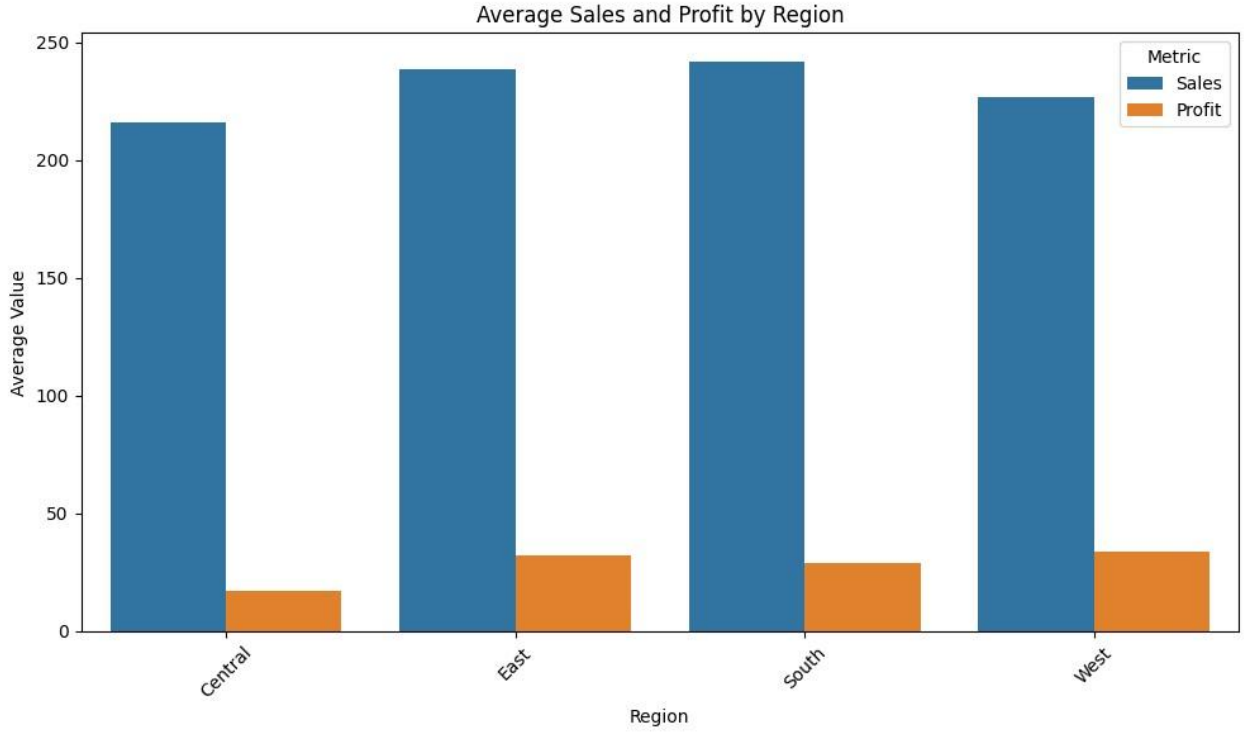
In [22]:

```
plt.figure(figsize=(17,8))sns.countplot(x="Sub-Category",      hue="Region",
data=
```



In [23]:

```
grouped_data = df.groupby('Region')[['Sales', 'Profit']].mean().reset_index()
```

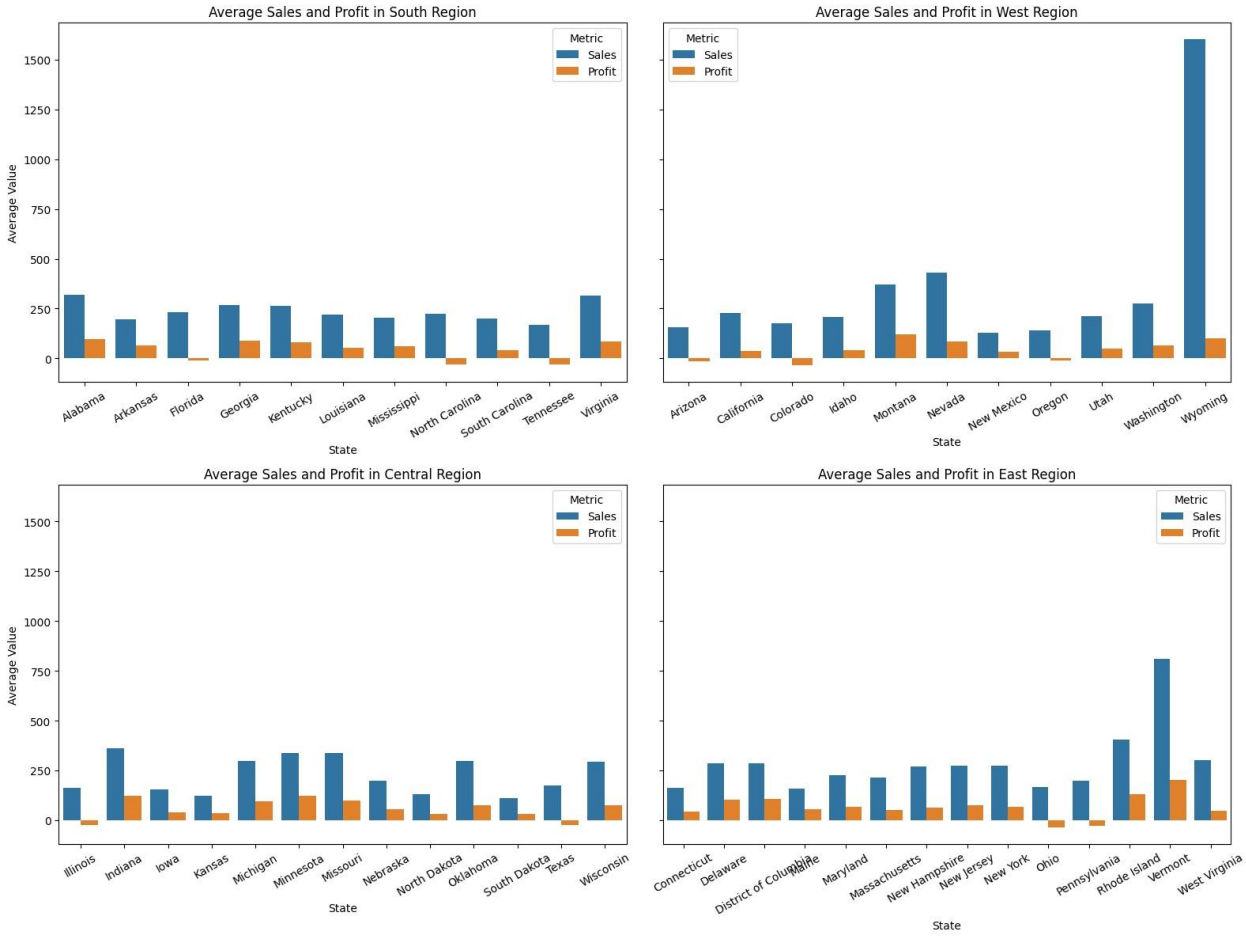


People in the west and east are more inclined to buy from department superstores than those in the south and center.People in the south have more expensive purchases.

with more details

In [24]:

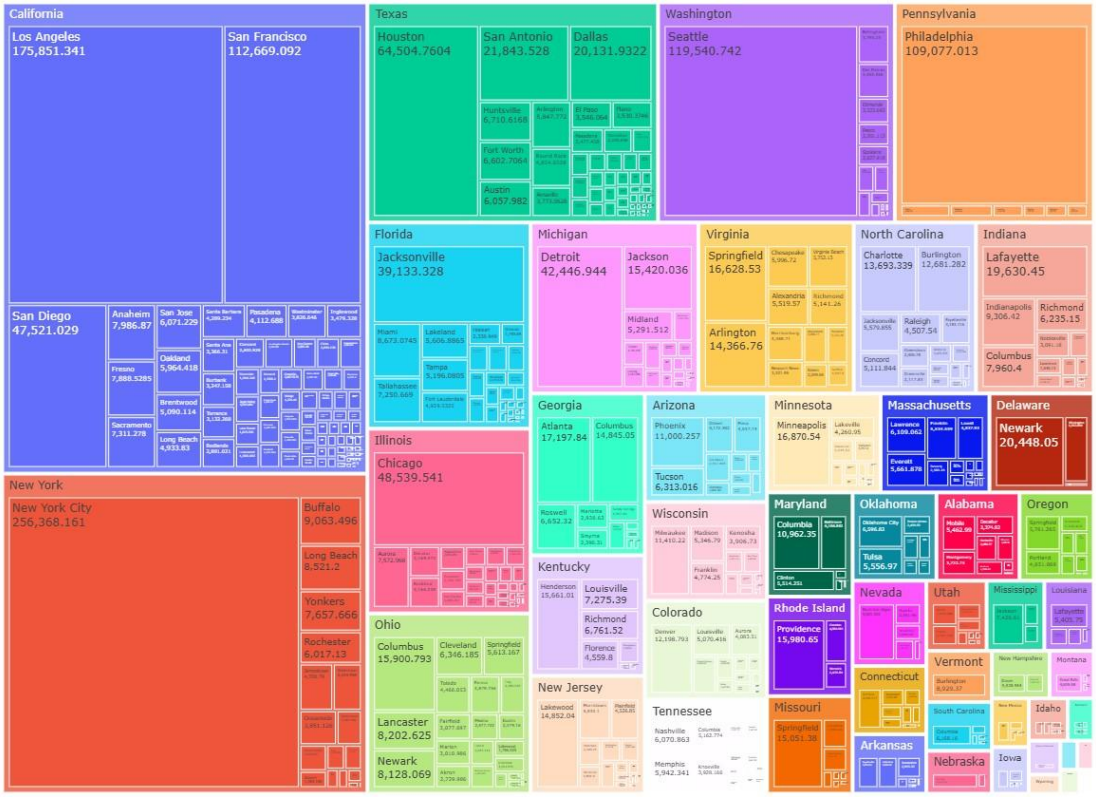
```
regions = df['Region'].unique()fig, axes = plt.subplots(2, 2, figsize=(16, 12
```



In [42]:

```
temp =
df[['State','City','Sales']].groupby(['State','City'])['Sales'].sum().
```

City-wise Sales

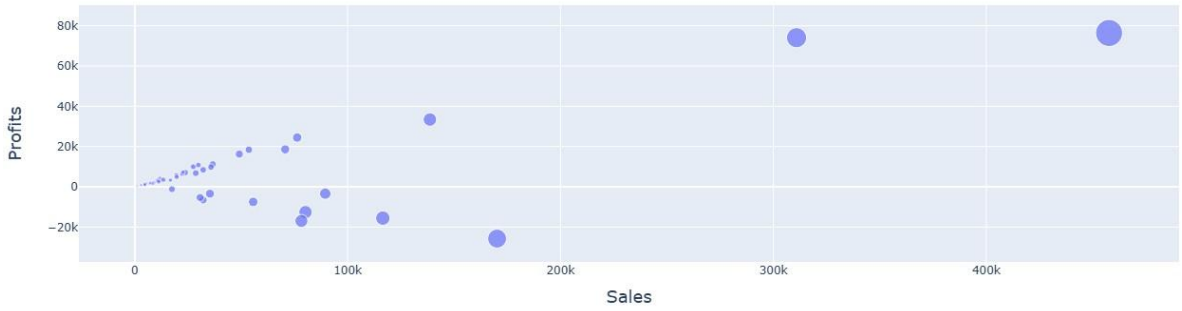


Above graph indicates->Which city and state has how much sales->Size of a tile indicates corresponding sales

In [28]:
```
df_notime = df[['Segment', 'City', 'State', 'Region', 'Category', 'Sub-Categor
```

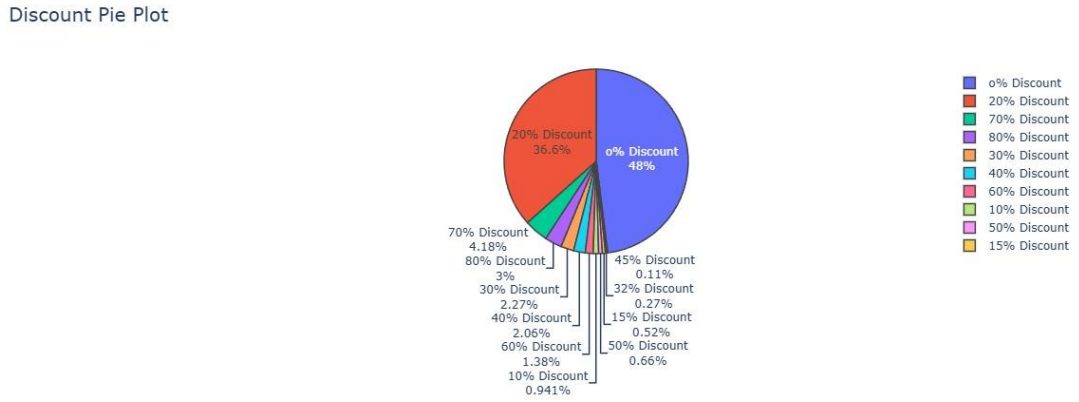
In [ ]:
```
fig =
go.Figure(data=px.scatter(x=df_notime.groupby('State').sum()['Sales'],
```

Sales & Profits by States

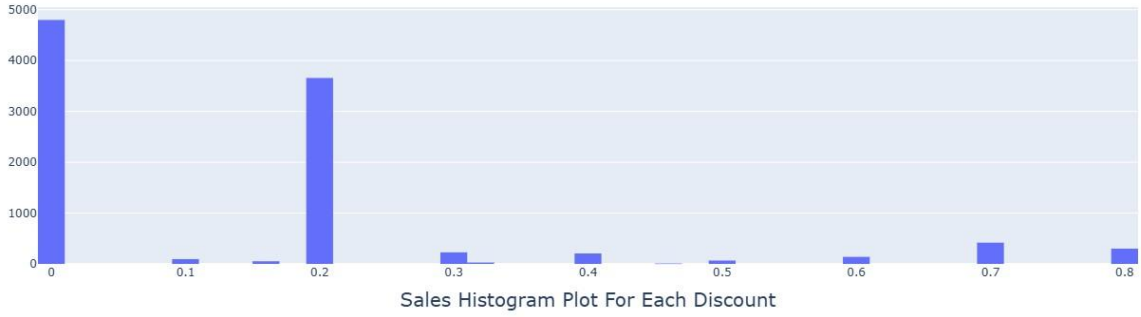


Discount Analysis

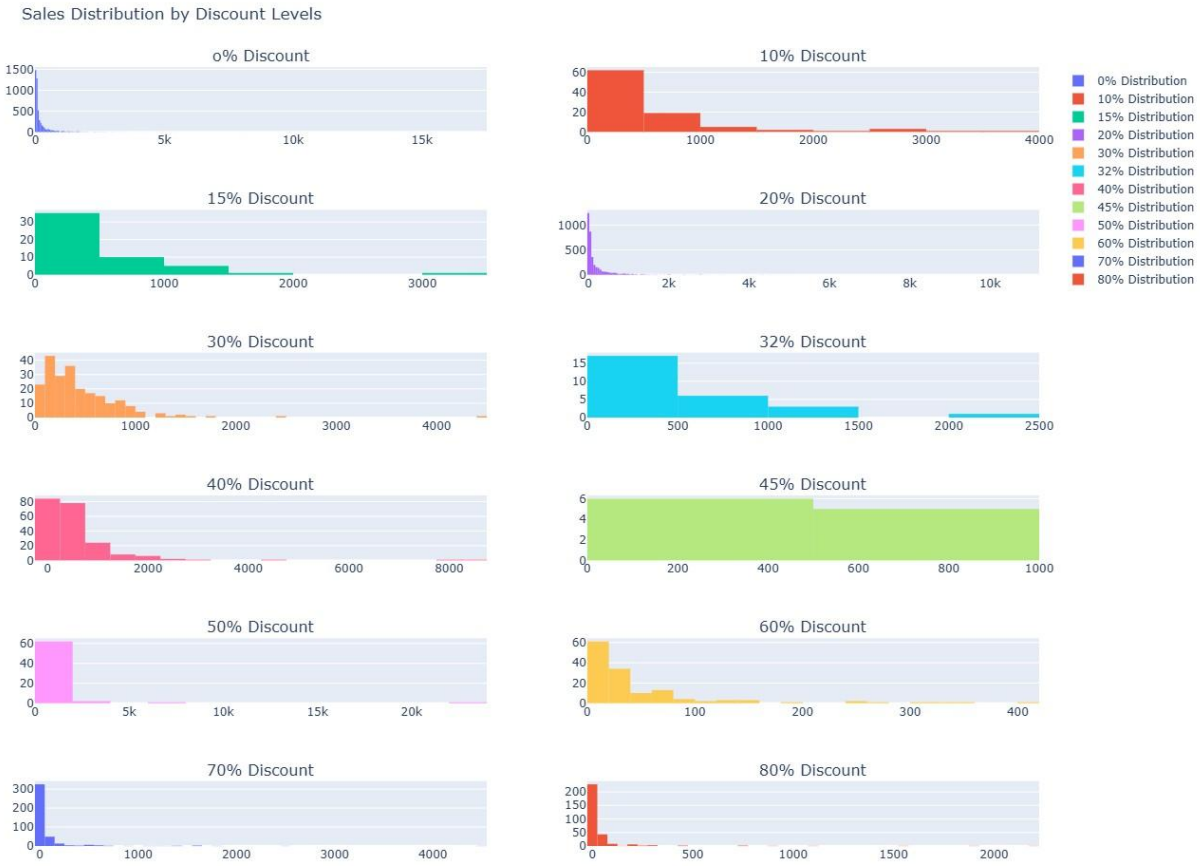
In [35]:
```
labels = ['o% Discount','20% Discount','70% Discount','80% Discount','30% Disc
```



In [36]:
```
data=go.Histogram(x=df_notime.sort_values('Discount')['Discount']),layout=go.
```



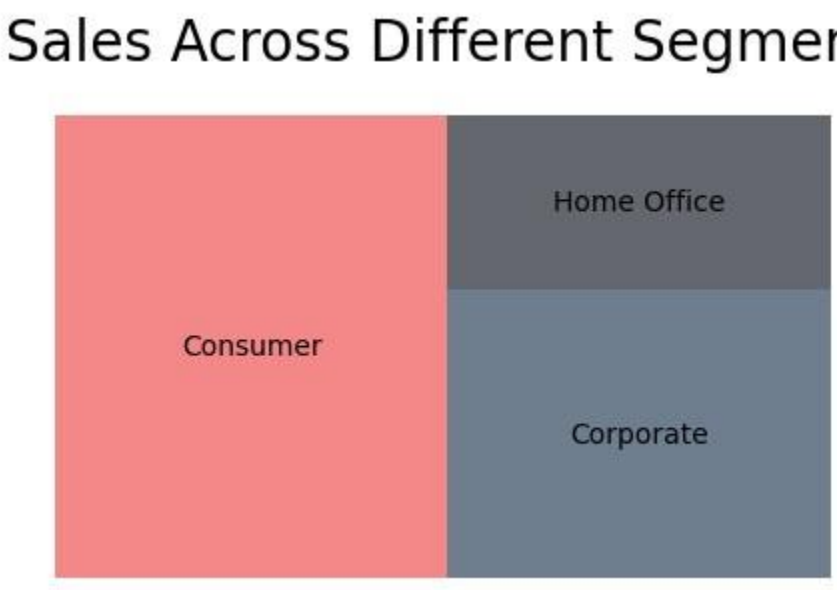
In [38]:
```
from plotly.subplots import make_subplotsfig = make_subplots(6,2,subplot_title
```



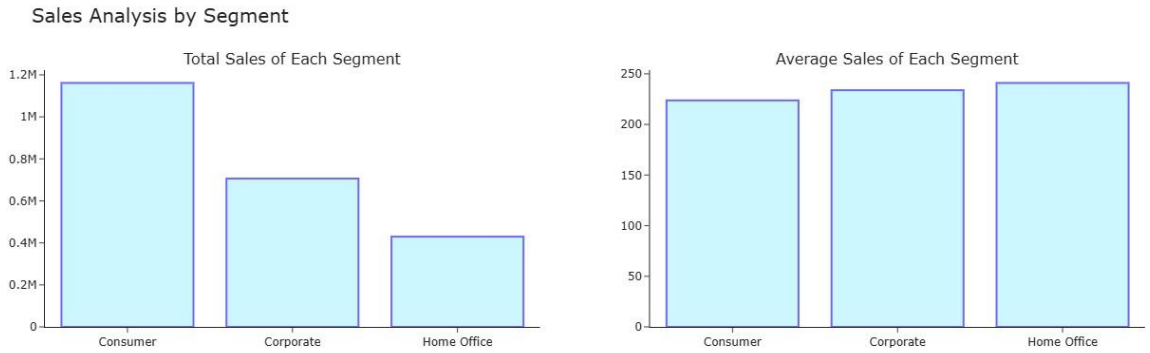
By applying the right policy on discounts, the loss of the store can be avoided in some sections.

Segmentation

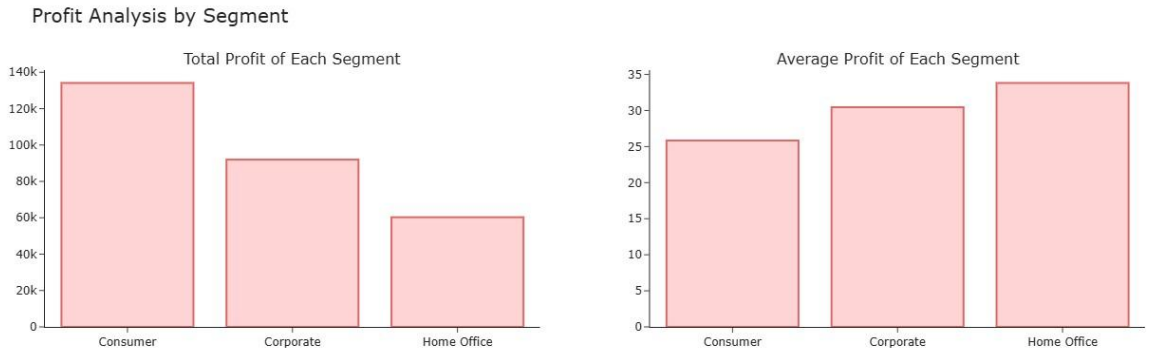
In [52]:
```
import squarifydf_tree = df[['Segment','Sales']]df_tree =
df_tree.groupby(['Se
```



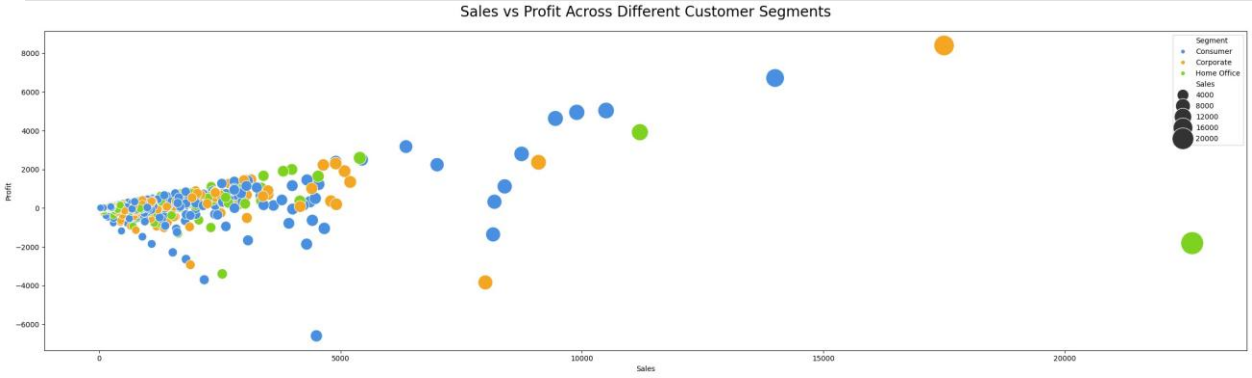
In [ ]:
```
segments = df['Segment'].unique()total_sales = [df[df['Segment'] == i]['Sales'
```



In [ ]:
```
total_profit = [df[df['Segment'] == i]['Profit'].sum() for i in segments]avera
```



In [71]:
```
df_scatter =
df[['Sales','Profit','Segment']].copy()plt.figure(figsize=[30,8])
```

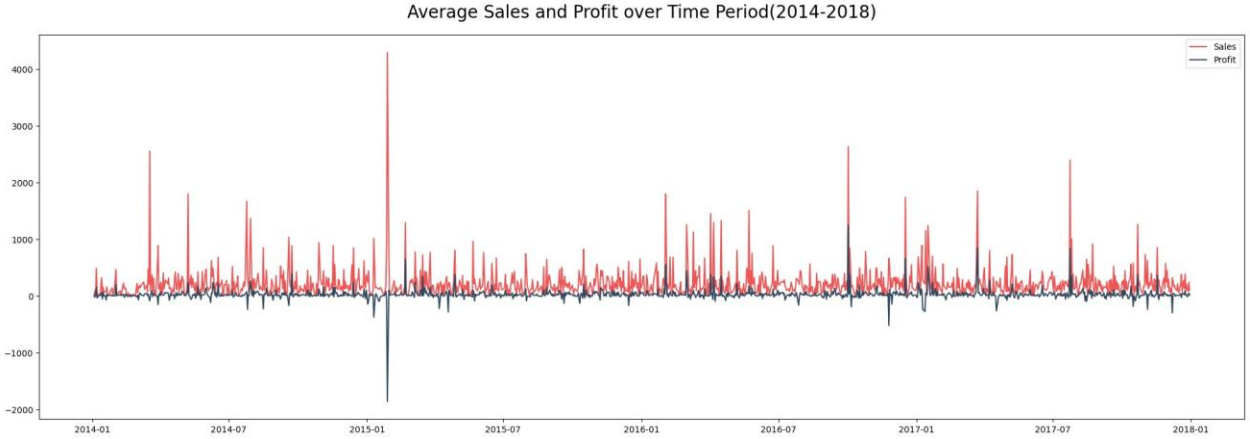


Sales:While the Consumer segment contributes to the highest total sales, the Home Office segment generates higher average sales per transaction. This suggests that consumers are more frequent buyers but tend to make smaller purchases, whereas home office customers, though fewer in number, make larger purchases on average.Profitability:Similarly, the Consumer segment has the highest total profit, but the Home Office segment achieves greater average profit per transaction. This indicates that the products or services sold to home office customers are more profitable on average.

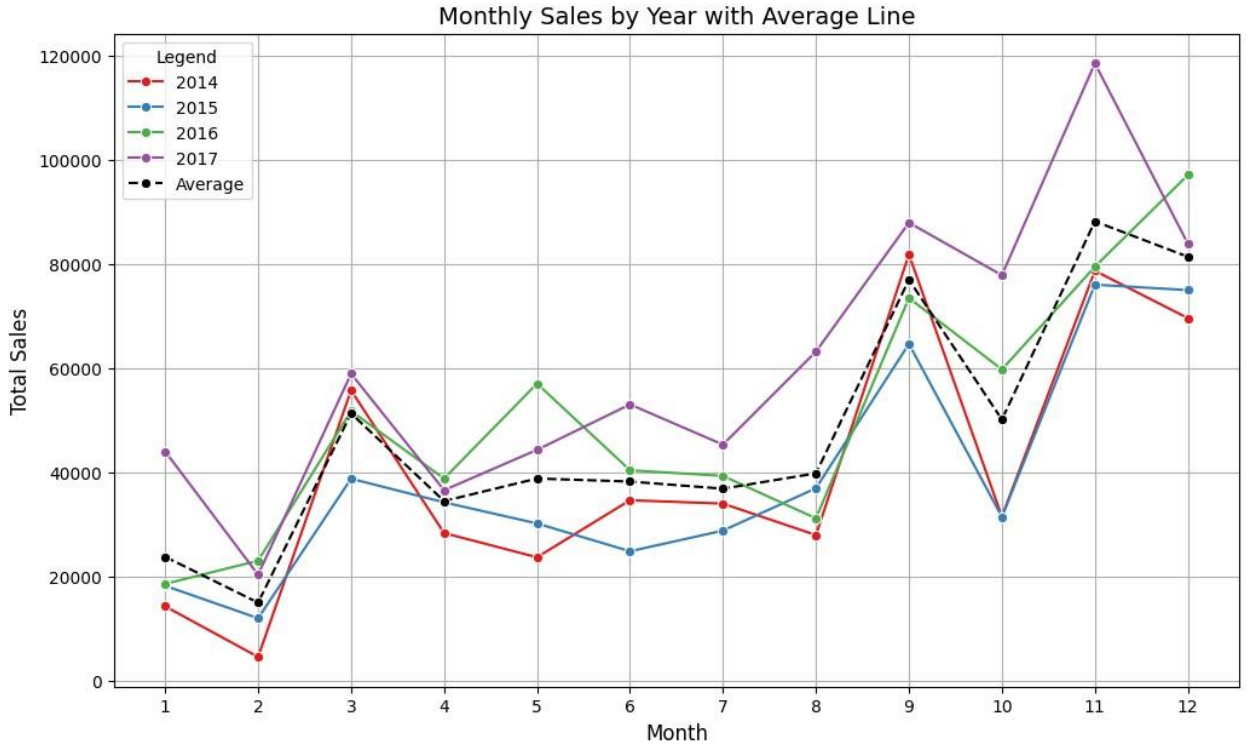
Analyze the Orders & Sales

In [58]:
```
df_time = df[['Order Date', 'Segment', 'Sales', 'Quantity', 'Discount', 'Profi
```

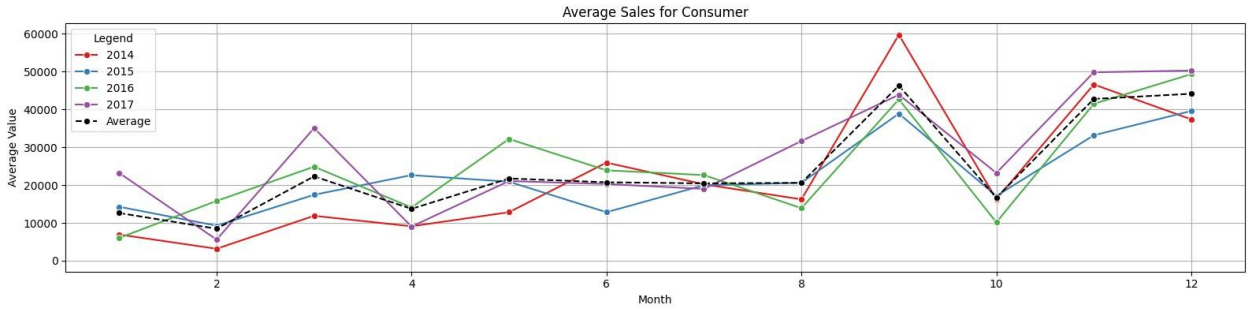
In [59]:
```
df_line = df_time[['Order Date','Sales','Profit']].sort_values('Order Date')
```

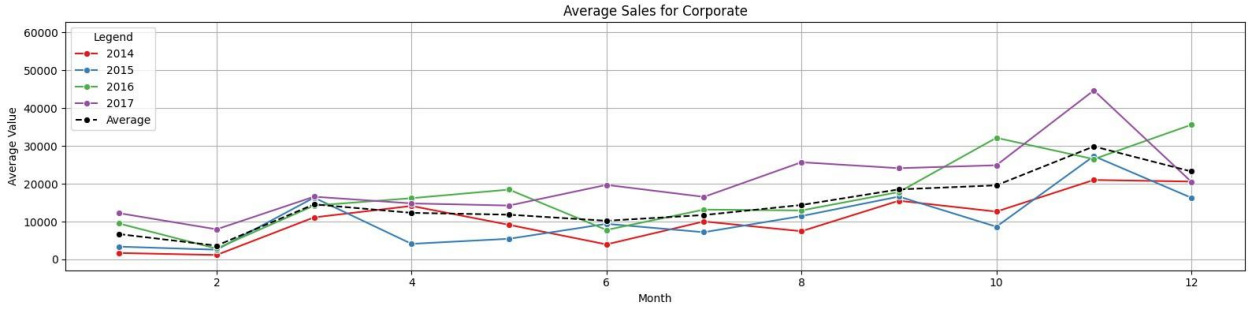


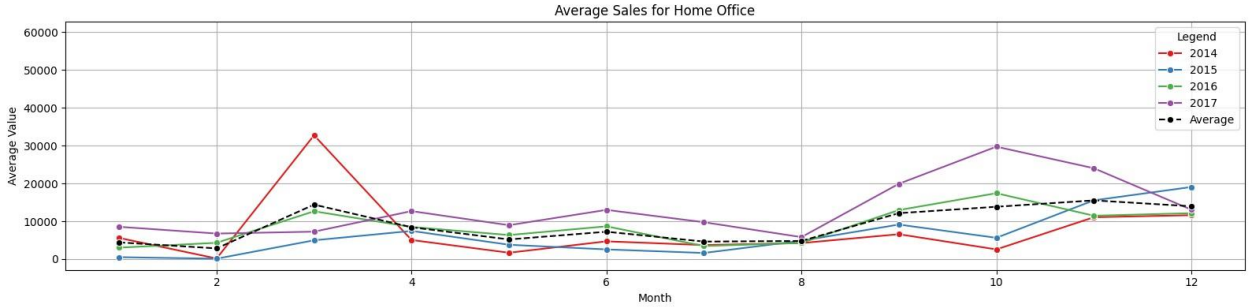
In [61]:
```
monthly_sales = df_time.groupby(['Year', 'Month'])['Sales'].sum().reset_index
```



In [72]:
```
fig, axes = plt.subplots(3, 1, figsize=(16, 12), sharey=True)axes = axes.flatt
```







In the store's sales data, customer purchases show a significant upward trend during specific months, namely March (3), September (9), November (11), and December (12). This pattern suggests seasonality, potentially influenced by holidays, promotions, or year-end activities. Conversely, there is a noticeable decline in purchases during January (1), February (2), and October (10), possibly due to post-holiday slowdowns or other factors like reduced promotional activities.On the other hand, purchases in the Home Office and Corporate segments are relatively consistent throughout the year, reflecting steady operational or business-driven demand. However, these segments also show a slight dip in January, likely due to slower business activities at the start of the year.

Thanks: